



# SentryCode

## Standalone Web UI - Product & System Requirements

Version 0.1 Draft

*Lightweight, professional engineering-compliance console for the standalone SentryCode product.*

|          |                                |
|----------|--------------------------------|
| Product  | QUILONS SentryCode             |
| Document | Standalone Web UI Requirements |
| Version  | 0.1 Draft                      |
| Date     | 29 August 2026                 |

# Executive Summary

This specification defines a lightweight standalone web UI for QUILONS SentryCode. The UI complements the existing CLI, Git/CI integrations and SentryCode service/API. It does not duplicate SentryCode policy or scanner logic, and it does not depend on QUILONS Compliance or QUILONS CRA.

The design is intentionally engineering-focused: fast to understand, information-dense, evidence-first, secure by default, and professional enough for enterprise security, DevOps and software-compliance teams.

## Contents

1. 1. Purpose
2. 2. Product Positioning
3. 3. Primary Users
4. 4. Design Principles
5. 5. Application Structure
6. 6. Overview Dashboard
7. 7. Repository Management
8. 8. Run History
9. 9. Run Details
10. 10. Findings
11. 11. Finding Details
12. 12. Vulnerability Management
13. 13. Dependency and License View
14. 14. SBOM Viewer
15. 15. Policy Management
16. 16. Enforcement Configuration
17. 17. Scanner Configuration
18. 18. Waiver Management
19. 19. Release Gate View
20. 20. Evidence Explorer
21. 21. Provenance and Attestation
22. 22. Git Assurance
23. 23. Automotive UI
24. 24. Integrations
25. 25. QUILONS Compliance Integration
26. 26. Settings
27. 27. Authentication
28. 28. Roles and Permissions
29. 29. Audit Requirements
30. 30. API Boundary
31. 31. Real-Time Behaviour
32. 32. Visual Design Requirements
33. 33. Responsive Behaviour
34. 34. Accessibility
35. 35. Performance
36. 36. Error Handling

- 37. 37. Security
- 38. 38. Technology Direction
- 39. 39. Initial v0.1 UI Scope
- 40. 40. Explicit Non-Goals
- 41. 41. UX Success Criteria
- 42. 42. Product Boundary
- 43. 43. Recommended Product Experience

# 1. Purpose

The QUILONS SentryCode Web UI SHALL provide a lightweight, professional, engineering-focused interface for operating SentryCode as a standalone product. The UI SHALL complement, not replace, the existing SentryCode CLI and CI/CD interfaces.

- Understand the current compliance/security state of repositories.
- Review SentryCode findings.
- Review merge and release-gate decisions.
- Configure scanners and enforcement policies.
- Manage controlled waivers.
- Inspect vulnerabilities, dependency/license issues and SBOMs.
- Review evidence, provenance and attestations.
- Configure repositories and integrations.
- Understand whether SentryCode itself is operational.

The UI SHALL remain usable without QUILONS Compliance or QUILONS CRA.

# 2. Product Positioning

SentryCode SHALL remain a standalone engineering-compliance product. Its primary interfaces SHALL be:

```
SentryCode
|- CLI
|- Git / CI integrations
|- Capability API / service
`- SentryCode Web UI
```

QUILONS Compliance integration SHALL remain optional.

```
SentryCode UI
|
SentryCode Service
|
`--- Optional ---> QUILONS Compliance
                    |
                    `--- CRA / Cyber / future modules
```

# 3. Primary Users

- Software developers
- DevOps engineers
- Platform engineers
- Application security engineers
- Security teams
- Engineering managers
- Software compliance engineers
- Release managers

The UI SHALL therefore prioritise engineering concepts rather than broader regulatory/GRC terminology.

## 4. Design Principles

### 4.1 Lightweight

The application SHALL avoid unnecessary visual complexity. The main operational state of SentryCode SHOULD be understandable within seconds of opening the application.

### 4.2 Professional

The visual design SHALL be appropriate for enterprise engineering and security environments and SHALL avoid consumer-style gamification, excessive animation, decorative illustrations, unnecessary gradients and oversized marketing components inside operational screens.

### 4.3 Information Dense but Clear

The UI SHOULD favour compact tables, clear status badges, filters, expandable details, contextual side panels, and summary cards only where useful.

### 4.4 Evidence First

A user SHALL always be able to move from decision to finding to scanner result to evidence to repository/commit/build.

### 4.5 CLI Parity

Anything represented in the UI SHALL map to an existing SentryCode capability, configuration contract or API operation where practical. The UI SHALL NOT contain an independent compliance decision engine.

### 4.6 Read-Only by Default

Viewing findings, evidence and scans SHALL be the dominant mode. Actions that change enforcement policy or grant waivers SHALL require deliberate user interaction.

## 5. Application Structure

The primary navigation SHOULD contain:

- Overview
- Repositories
- Runs
- Findings
- Dependencies
  - | - Vulnerabilities
  - | - Licenses
  - ` - SBOM
- Policies
- Waivers
- Release Gates
- Evidence
- Automotive [only when enabled]
- Integrations
- Settings

The UI SHOULD hide sections that are not enabled by configuration. Automotive SHALL not appear unless the automotive capability is enabled.

## 6. Overview Dashboard

The Overview SHALL provide a concise operational summary.

### Current state

- Overall SentryCode health
- Number of monitored repositories
- Latest scan status
- Failing repositories
- Warning repositories
- Passing repositories

### Findings summary

- Critical findings
- High findings
- Medium findings
- Low findings

### Engineering compliance summary

- Secret violations
- Vulnerability violations
- License violations
- SAST violations
- Git assurance violations
- Provenance failures

### Recent activity

- Latest scans
- Latest release gates
- Recent waivers
- Policy changes
- Evidence publication events

The dashboard SHALL NOT imply that an organisation is legally compliant merely because SentryCode reports PASS.

## 7. Repository Management

The Repositories screen SHALL show configured repositories/projects.

- Repository name
- Repository identifier
- Default branch
- Latest commit scanned
- Latest scan status
- Latest release-gate status
- Number of active findings

- Last scan time
- Policy profile
- CI integration state

A repository details screen SHALL provide Repository Overview, Latest Run, Findings, Dependencies, SBOM, Policies, Waivers, Release History, Evidence and Configuration.

## 8. Run History

The Runs screen SHALL show SentryCode executions with Run ID, repository, commit SHA, branch, trigger, start time, duration, scanner status, policy result, PASS/WARN/FAIL and evidence integrity status.

Supported trigger labels SHOULD include Local CLI, Git hook, Pull request, CI pipeline, Manual and Release gate. Users SHALL be able to filter runs by repository, branch, status, date, scanner and trigger.

## 9. Run Details

A run details page SHALL explain exactly why SentryCode reached its decision.

```
Run #SC-1028
Result: FAIL
Repository: payments-service
Commit: a8f34de
Branch: feature/payment-update
Policy: production-standard-v3

Scanners
[PASS] Secrets
[PASS] Dependencies
[PASS] SAST
[PASS] Git Assurance
[PASS] Provenance

Findings
2 Critical
1 High
4 Medium
```

The page SHALL expose scanner outputs, normalized findings, policy evaluation, waivers applied, evidence generated, scanner failures, configuration metadata and version metadata.

## 10. Findings

The Findings screen SHALL provide a unified view over normalized SentryCode findings.

- Severity
- Finding type
- Rule ID
- Title
- Repository
- Service
- File/package
- Scanner

- Policy result
- Waiver status
- First detected
- Last detected

Filters SHOULD include severity, repository, scanner, finding type, policy result, waived/unwaived state, service/package and date. Selecting a finding SHALL open a detail view.

## 11. Finding Details

### Finding

- Title
- Severity
- Description
- Rule ID
- Scanner
- Source

### Location

- Repository
- Commit
- File
- Line/range
- Service
- Package/component

### Enforcement

- Policy result
- Blocking/non-blocking status
- Active waiver
- Relevant policy rule

### Remediation

- Recommended remediation
- Fixed version where applicable
- References where available

### Evidence

- Evidence ID
- Scan/run ID
- Timestamp
- Evidence integrity status

Secrets SHALL always remain redacted. The UI SHALL NEVER display full detected secret values.



## 12. Vulnerability Management

- Vulnerability ID
- Package
- Installed version
- Fixed version
- Severity
- Known exploit status where available
- Repository
- Service
- Policy result
- Waiver state
- Vulnerability intelligence source
- Intelligence database/version

Users SHOULD be able to identify newly introduced, existing and fixed vulnerabilities.

## 13. Dependency and License View

The Dependencies view SHALL provide visibility into dependencies, package versions, registries, license information, policy state and vulnerability state.

Allowed  
Review Required  
Denied  
Unknown  
Waived

The UI SHALL NOT present a particular licence as universally legally acceptable or unacceptable. The displayed result SHALL reflect the organisation's configured policy.

## 14. SBOM Viewer

The UI SHALL provide a human-readable representation of generated SBOMs and SHOULD support package search, component filtering, version inspection, license inspection, vulnerability correlation and repository/commit provenance. Users SHOULD be able to download the original machine-readable SBOM artifact. Initial supported formats: CycloneDX and SPDX.

## 15. Policy Management

Policy configuration is a core UI capability. The Policies screen SHALL expose SentryCode's policy hierarchy:

Organisation  
|  
Tenant  
|  
Project  
|  
Repository  
|  
Service

The UI SHALL clearly show inherited values, locally configured values, locked parent values and the effective resulting policy.

## 16. Enforcement Configuration

Users SHALL be able to configure whether findings ALLOW, WARN or FAIL. The UI SHOULD use clear engineering language such as Block, Block unless waived, Warn and Allow. The configuration SHALL map to SentryCode policy contracts. The UI SHALL NOT independently determine merge status.

## 17. Scanner Configuration

The UI SHOULD expose enabled scanners and their state.

- Secrets
- Dependencies
- Vulnerabilities
- Native SAST
- External SAST
- Git Assurance
- Provenance
- Automotive

For each scanner, the UI SHOULD show enabled/disabled, required/optional, failure behaviour, scanner version, configuration summary and latest operational result. Failure behaviour SHALL support Fail, Warn and Ignore where permitted by policy.

## 18. Waiver Management

The Waivers screen SHALL allow authorised users to review and manage temporary policy exceptions.

- Finding/rule
- Repository/service scope
- Reason
- Requester
- Approver
- Ticket/reference
- Created
- Expiration
- Status
- Associated findings

Statuses SHOULD include Pending, Active, Expired, Rejected and Revoked. Expired waivers SHALL be visually obvious. The underlying finding SHALL remain visible even when waived.

## 19. Release Gate View

Release Gates SHALL have a dedicated view. Each result SHOULD show repository, commit/tag, build, policy version, PASS/WARN/FAIL, blocking findings, applied waivers, evidence generated and timestamp.

RELEASE BLOCKED

2 policy requirements failed:

- Critical vulnerability in package xyz
- Build provenance attestation missing

## 20. Evidence Explorer

The Evidence screen SHALL expose SentryCode evidence records and SHOULD allow filtering by evidence type.

```
sbom.generated
license.check
dependency.check
vuln.scan
secret.scan
policy.eval
commit.verification
provenance.attestation
build.attestation
release.gate
```

Each evidence item SHALL show Evidence ID, schema version, repository, commit, run, timestamp, evidence type, integrity status and signature status where applicable. Users SHOULD be able to inspect the machine-readable representation.

## 21. Provenance and Attestation

- Repository
- Commit
- Branch/tag
- Builder
- Build ID
- Build timestamp
- Toolchain
- Artifact hash
- SBOM reference
- Signature verification result

The UI SHOULD clearly distinguish Verified, Unsigned, Invalid and Unknown.

## 22. Git Assurance

- Commit signature state
- Author identity state
- Protected branch state
- Required approval expectations
- Required status checks
- Repository governance status

Provider-specific capabilities SHALL only appear when available. GitHub governance information MAY appear where configured while other providers retain generic Git assurance.

## 23. Automotive UI

Automotive functionality SHALL appear only when enabled. The screen MAY contain Automotive Overview, MISRA Findings, AUTOSAR Findings, Deviations, Evidence Mapping and Software Update Provenance.

The UI SHALL NOT reproduce copyrighted MISRA or AUTOSAR rule text unless properly licensed. It SHALL NOT claim UNECE R155 compliance solely based on SentryCode checks; it SHOULD describe relevant engineering evidence instead.

## 24. Integrations

- Local Git hooks
- GitHub
- GitLab
- Azure DevOps
- Jenkins
- Generic CI
- External SAST
- Vulnerability intelligence
- QUILONS Compliance

Future integrations MAY include Gerrit and additional Git governance providers. For each integration, the UI SHOULD show Configured, Connected, Error or Not configured.

## 25. QUILONS Compliance Integration

SentryCode SHALL remain operational with this integration disabled. When configured, the UI SHOULD show Compliance connection state, plugin registration state, last evidence publication, tenant/project scope, authentication health and last publication error. The UI SHALL NOT expose secrets or raw service credentials.

## 26. Settings

### General

- SentryCode version
- Instance identifier
- Runtime mode
- Data location

### Security

- Evidence signing state
- Trusted public keys
- Configuration-signing state
- API exposure

### Intelligence

- Vulnerability database state
- Last update

- Source
- Offline bundle version

## Storage

- Evidence location
- Retention
- Backup status

## Diagnostics

- Health
- Readiness
- Scanner availability
- External integrations

# 27. Authentication

The standalone SentryCode UI SHALL require authentication when accessed over anything other than a trusted local development context. The implementation SHOULD support an extensible authentication boundary suitable for Local administrator mode, Enterprise OIDC and QUILONS identity integration where available. Authentication SHALL be separate from tenant/project capability credentials used between SentryCode and QUILONS Compliance.

# 28. Roles and Permissions

## Viewer

- View runs
- View findings
- View evidence
- View SBOMs
- View policies

## Engineer

- View engineering information
- Request waivers
- Configure repository-level settings where permitted

## Security / Policy Administrator

- Change policy
- Configure scanners
- Approve/reject waivers
- Configure enforcement

## Administrator

- Configure SentryCode instance
- Configure integrations
- Manage security/trust settings

Exact role implementation MAY evolve.

## 29. Audit Requirements

Administrative changes made through the UI SHALL use the same governed SentryCode audit model as changes made through other supported interfaces.

- Policy modification
- Scanner configuration change
- Waiver creation
- Waiver approval/rejection
- Integration changes
- Trust/security configuration changes

The UI SHALL NOT introduce an unaudited configuration path.

## 30. API Boundary

The UI SHALL communicate with SentryCode through versioned application/service APIs and SHALL NOT directly access SentryCode storage/database internals.

```

Browser
|
SentryCode Web Application
|
SentryCode API / Application Service
|
Existing SentryCode Core
|- scanners
|- policy
|- evidence
|- waivers
|- repository
|- integrations
  
```

Existing business logic SHALL be reused.

## 31. Real-Time Behaviour

The UI SHOULD provide timely updates for active scans. Heavy WebSocket infrastructure SHALL NOT be required for v0.1. Acceptable mechanisms include lightweight polling, server-sent events, or existing governed event infrastructure where appropriate. The UI SHOULD display Queued, Running, Completed and Failed for applicable operations.

## 32. Visual Design Requirements

- Light professional default theme
- Optional dark mode
- Neutral enterprise visual language
- Restrained status colours
- Compact navigation
- Clear typography

- Subtle borders
- Minimal shadows

Primary status semantics SHOULD be PASS, WARN, FAIL and UNKNOWN. Colour SHALL NOT be the only method used to communicate status; icons/text labels SHALL reinforce meaning.

## 33. Responsive Behaviour

The primary target SHALL be desktop and laptop browsers. Minimum recommended design target: 1280 x 720. The UI SHOULD remain usable on tablets. Full mobile administration SHALL NOT be a v0.1 requirement.

## 34. Accessibility

The UI SHOULD target WCAG 2.1 AA-compatible design principles.

- Keyboard-accessible navigation
- Visible focus indicators
- Semantic HTML
- Sufficient colour contrast
- Labels in addition to colours/icons
- Accessible tables/forms
- Screen-reader-friendly status text

## 35. Performance

- Initial application shell visible quickly.
- Navigation transitions without full application reload.
- Tables paginated or virtualised where necessary.
- Large evidence/SBOM objects loaded on demand.
- Filtering performed efficiently.

The UI SHALL NOT attempt to render extremely large SBOMs or finding sets into a single unbounded DOM view.

## 36. Error Handling

Policy failure  
Scanner failure  
Configuration error  
Authentication failure  
Integration failure  
Infrastructure failure  
Evidence integrity failure

A SentryCode scanner/runtime failure SHALL never visually appear as PASS. Evidence integrity failures SHALL be prominent, e.g. "Evidence verification failed. This run cannot currently be treated as trusted evidence."

## 37. Security

- Never expose full detected secrets.
- Avoid source-code transmission outside SentryCode unless explicitly configured.
- Avoid embedding service credentials in browser-visible configuration.

- Use secure session handling.
- Protect state-changing operations against CSRF where applicable.
- Validate all API input.
- Use least privilege.
- Preserve tenant/project isolation where enabled.
- Respect evidence integrity verification.
- Avoid direct repository modification except for deliberate operations.

## 38. Technology Direction

The UI SHOULD be implemented as a small modern web application rather than a large independent platform. Recommended architecture: React / TypeScript plus a small SentryCode web backend/API extension, or an equivalent lightweight technology. Existing SentryCode TypeScript contracts SHOULD be reused wherever feasible. A large microservice architecture SHALL NOT be introduced solely for the UI.

## 39. Initial v0.1 UI Scope

### Phase 1 - Essential Standalone UI

- Application shell/navigation
- Overview dashboard
- Repository list/details
- Run history/details
- Findings
- Vulnerabilities
- Dependency/license view
- Policy viewer/editor
- Scanner configuration
- Waiver management
- Release gates
- Evidence explorer
- Health/readiness
- Basic integration status

### Phase 2 - Enhanced Engineering Views

- SBOM explorer
- Provenance viewer
- Git governance UI
- Automotive UI
- Vulnerability intelligence management
- Backup/retention UI
- Compliance integration settings

## 40. Explicit Non-Goals for UI v0.1

- Source-code editor
- Git hosting platform
- CI/CD system



- Generic issue tracker
- SIEM
- EDR console
- Penetration-testing product
- Generic GRC dashboard
- QUILONS Compliance UI
- CRA UI

The UI focus SHALL remain engineering compliance and security decisions around source changes, builds and releases.

## 41. UX Success Criteria

A developer or security engineer SHOULD be able to answer the following within approximately one minute of opening SentryCode:

- Are our repositories currently passing SentryCode policy?
- What is blocking merge or release?
- Which critical/high findings exist?
- Which repository/commit introduced the issue?
- What policy caused it to block?
- Is there an approved waiver?
- What evidence was produced?
- Did all required scanners actually run?

A policy/security administrator SHOULD be able to:

- Change a permitted enforcement rule.
- Configure block/warn/allow behaviour.
- Approve or reject a waiver.
- Determine policy inheritance.
- Verify that required scanners are operational.
- Review an auditable record of those changes.

## 42. Product Boundary

The SentryCode UI answers:

*“What is happening in our continuous engineering-compliance checks, why is a change/build/release passing or failing, and how is SentryCode configured to enforce our engineering policy?”*

QUILONS Compliance answers the broader question:

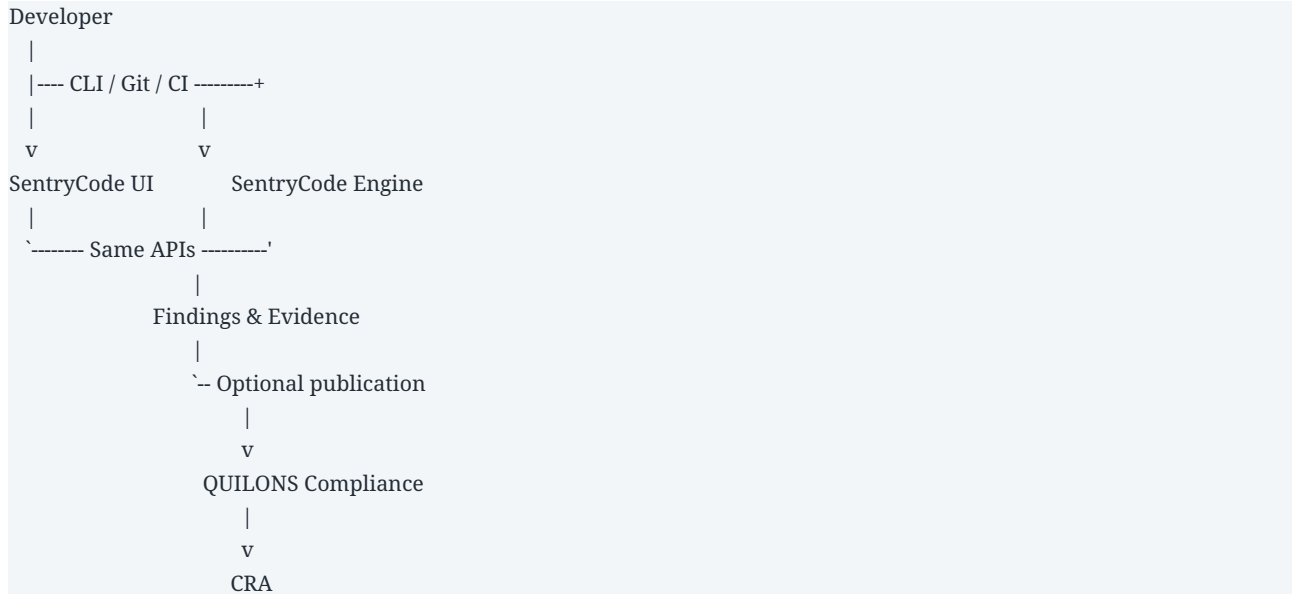
*“What compliance capabilities and evidence exist across the organisation and its installed compliance modules?”*

QUILONS CRA answers:

*“Can the organisation demonstrate that relevant Cyber Resilience Act requirements are being satisfied?”*

These responsibilities SHALL remain separate.

## 43. Recommended Product Experience



This preserves SentryCode as a complete standalone product while allowing the broader QUILONS platform to consume its evidence when required.

## Implementation Recommendation

Phase 1 SHOULD be the implementation target. Keep the first implementation deliberately small: one web application, one existing SentryCode backend/service boundary, no new microservices, and no duplicated policy logic. This is sufficient to make SentryCode a credible standalone enterprise product without turning the UI into another platform project.